

ADVANCED OPERATING SYSTEMS

Lab 5 — Writing a UART driver for Linux

Vittorio Zaccaria

Politecnico di Milano

Overview

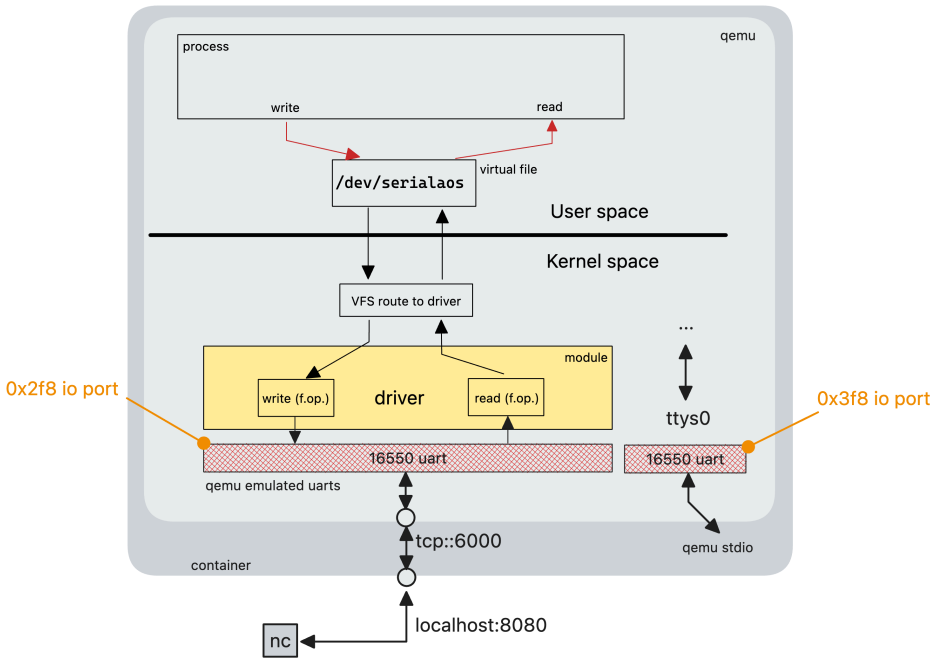
What you will build

One character driver for the second PC serial port (COM2: a 16550 UART at I/O ports `0x2F8–0x2FF`, IRQ 3), written in two stages, then used for real.

- **Part A — polled transmit.** Claim the ports, program the UART, register `/dev/serialaos`, and send bytes by polling a status bit.
- **Part B — interrupt-driven receive.** Handle IRQ 3, queue received bytes in a ring buffer, and make `read` sleep until data arrives.
- **Part C — use the driver.** Profile your own driver with `perf` and carry the profile out of the guest through `/dev/serialaos`.

Each stage ends with a build and a **checkpoint** in QEMU. Answer the questions on each checkpoint slide before moving on.

The driver we are going to build



Set up the module and COM2

```
# Host
mkdir -p modules/lab-5-work
printf '%s\n' 'include $(dir $(lastword $(MAKEFILE_LIST)))../common.mk' \
  > modules/lab-5-work/Makefile
```

Write `modules/lab-5-work/module.c` from the Lab 1 hello module. You need two host terminals:

```
# Terminal 1: guest console, QEMU with COM2 on TCP port 6000
T=amd64 E=full make dev-up          # once
T=amd64 E=full make dev-build       # after every change
T=amd64 E=full make UART2=6000 dev-run
# Terminal 2: the "other end of the cable"
T=amd64 E=full make UART2=6000 dev-uart2
```

Terminal 2 shows what the guest sends on COM2 and sends each line you type. Close it with `Ctrl-C`. Browser frontend: tasks AOS: Run QEMU with COM2 on port 6000 (1 vCPU) **and** AOS: Connect to COM2 (port 6000). Exit QEMU with `Ctrl-]`, then `x`.

The guest kernel registers only `ttyS0` (the console), so COM2 is free for you.

Part A — Polled transmit

The 16550 UART

0x5 enable RDA and RLS

0x0 disable hw FIFO

	REGISTER ADDRESS											
Bit No.	0 DLAB=0	0 DLAB=0	1 DLAB=0	2		3	4	5	6	7	0 DLAB=1	1 DLAB=1
	Receiver Buffer Register (Read Only)	Transmitter Holding Register (Write Only)	Interrupt Enable Register	Interrupt Ident. Register (Read Only)	FIFO Control Register (Write Only)	Line Control Register	MODEM Control Register	Line Status Register	MODEM Status Register	Scratch Register	Divisor Latch (LS)	Divisor Latch (MS)
	RBR	THR	IER	IIR	FCR	LCR	MCR	LSR	MSR	SCR	DLL	DLM
0	Data Bit 0[1]	Data Bit 0	Enable Received Data Available Interrupt (ERBFI)	"0" if Interrupt Pending	FIFO Enable	Word Length Select Bit 0 (WLS0)	Data Terminal Ready (DTR)	Data Ready (DR)	Delta Clear to Send (DCTS)	Bit 0	Bit 0	Bit 8
1	Data Bit 1	Data Bit 1	Enable Transmitter Holding Register Empty Interrupt (ETBEI)	Interrupt ID Bit (0)	RCVR FIFO Reset	Word Length Select Bit 1 (WLS1)	Request to Send (RTS)	Overrun Error (OE)	Delta Data Set Ready (DDSR)	Bit 1	Bit 1	Bit 9
2	Data Bit 2	Data Bit 2	Enable Receiver Line Status Interrupt (ELSI)	Interrupt ID Bit (1)	XMIT FIFO Reset	Number of Stop Bits (STB)	Out 1	Parity Error (PE)	Trailing Edge Ring Indicator (TERI)	Bit 2	Bit 2	Bit 10
3	Data Bit 3	Data Bit 3	Enable MODEM Status Interrupt (EDSSI)	Interrupt ID Bit (2) [2]	DMA Mode Select	Parity Enable (PEN)	Out 2	Framing Error (FE)	Delta Data Carrier Detect (DDCD)	Bit 3	Bit 3	Bit 11
4	Data Bit 4	Data Bit 4	0	0	Reserved	Even Parity Select (EPS)	Loop	Break Interrupt (BI)	Clear to Send (CTS)	Bit 4	Bit 4	Bit 12
5	Data Bit 5	Data Bit 5	0	0	Reserved	Stick Parity	0	Transmitter Holding Register (THRE)	Data Set Ready (DSR)	Bit 5	Bit 5	Bit 13
6	Data Bit 6	Data Bit 6	0	FIFOs Enabled [2]	RCVR Trigger (LSB)	Set Break	0	Transmitter Empty (TEMT)	Ring Indicator (RI)	Bit 6	Bit 6	Bit 14

0x4 = character received

1 if the THR reg is free.

The 16550 registers

OFFSET	DLAB	READ	WRITE
0	0	RBR: received byte	THR: byte to send
0	1	DLL: baud divisor, low byte	same
1	0	IER: interrupt enable	same
1	1	DLM: baud divisor, high byte	same
2	—	IIR: interrupt identification	FCR: FIFO control
3	—	LCR: line control (bit 7 = DLAB)	same
4	—	MCR: modem control	same
5	—	LSR: line status (bit 5 = THRE)	—

Addresses are `0x2F8 + offset`. Baud rate is `115200 / divisor`. Full details: 16550 datasheet.

A1 · Task: claim the ports, register the device

Use one `#define BASE 0x2F8` and address registers as `BASE + offset`. In init:

1. reserve the 8 ports with `request_region`; if they are busy return `-EBUSY`;
2. **then** register a character device named `serialaos` with a dynamic major and print `serialaos: major=<major>`; on failure release the ports.

Exit undoes both in reverse order. QEMU ignores baud rate and line format, and the FIFO is off after reset: you do not program them here (see question 7).

NEED	API
reserve / release I/O ports	`request_region`, `release_region` (`<linux/ioport.h>`)
port I/O	`inb(port)`, `outb(value, port)` (`<linux/io.h>`)
character device, dynamic major	`register_chrdev(0, name, &fops)`, `unregister_chrdev`

A2 · Task: a polled `write`

Fill a `struct file_operations` with `.owner = THIS_MODULE` and `.write`. For each byte of the user buffer:

1. fetch it with `get_user(c, buf + i)`; stop on failure;
2. busy-wait until `LSR` bit 5 (`0x20` , THRE: transmit holding register empty) is 1, calling `cpu_relax()` in the loop;
3. write it to `THR` .

Return the number of bytes sent, or `-EFAULT` if not even the first byte of a non-empty request could be read. Serialize concurrent writers with a `DEFINE_MUTEX` : `get_user` may sleep on a page fault.

A · Checkpoint

```
# Guest
grep -c 02f8 /proc/ioports
insmod /modules/lab-5-work.ko
grep serialaos /proc/ioports /proc/devices
M=$(awk '$2=="serialaos" {print $1}' /proc/devices); echo major=$M
mknod -m 600 /dev/serialaos c $M 0
echo "hello from the guest" > /dev/serialaos; echo write_status=$?
cat /dev/serialaos; echo read_status=$?
rmmod module
grep -c serialaos /proc/ioports
```

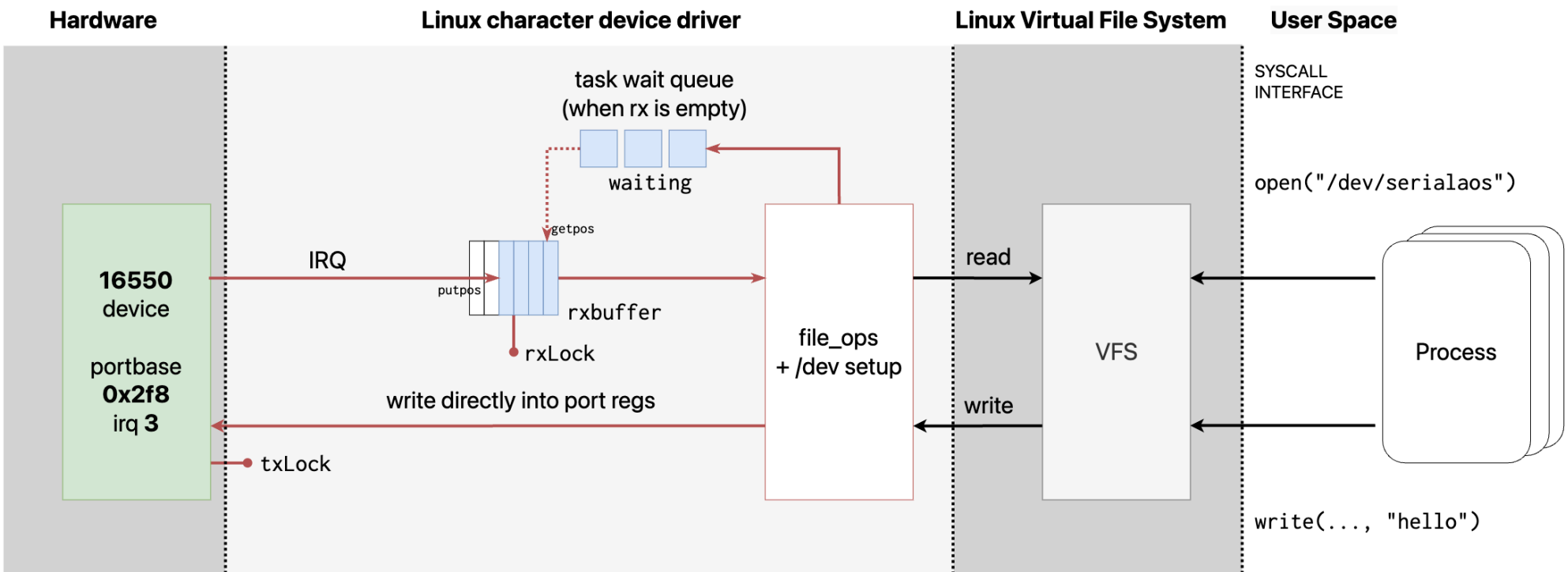
Expected: `0`, then `02f8-02ff : serialaos` and a major (248 in our run), `write_status=0`, the text in terminal 2, `cat: read error: Invalid argument` (no `.read` yet), `0` after `rmmod`.

A · Questions

1. What does `request_region` prevent? Does it stop anyone from calling `inb` / `outb` on those ports?
2. Why must the guest kernel register only one 8250 port for this lab to work?
3. Why must you redo `mknod` after every `insmod`?
4. Why a mutex and not a spinlock in `write`?
5. Where do the bytes go when nothing is connected to terminal 2?
6. How much CPU does polling cost at a real 115200 baud?
7. What would a driver for real hardware have to program before sending?

Part B — Interrupt-driven receive

Driver architecture



Serialaos driver as implemented during the lab, using only low-level character device primitives

Figure 1: lab 5 driver

B1 · Task: an interrupt handler

REGISTER	VALUE	MEANING
`IER` (`BASE + 1`)	`0x01`	`interrupt on received data`
`IIR` (`BASE + 2`) & `0x0f`	`0x04`	`received data available: read RBR`
`MCR` (`BASE + 4`)	`0x08`	`OUT2: routes the UART interrupt to the PC IRQ line`

- Add a 64-byte `DEFINE_KFIFO(fifo, char, 64)`, a `DECLARE_WAIT_QUEUE_HEAD(wq)` and a `dropped` counter.
- The handler returns `IRQ_NONE` unless `IIR` says “data available”; otherwise it `kfifo_put`s the byte from `RBR` (counting it in `dropped` if the fifo is full), wakes readers and returns `IRQ_HANDLED`.
- In init, after `request_region`: `request_irq(3, handler, 0, "serialaos", &fifo)`; free it if `register_chrdev` fails. Enable the interrupt (`MCR`, then `IER`) only after everything succeeded. In exit: unregister, `IER=0`, `free_irq`, release, and print `dropped=<n>`.

B2 · Task: a blocking `read`

Add `.read`:

- sleep until the fifo is not empty with `wait_event_interruptible`; if a signal interrupted the wait, return `-ERESTARTSYS`;
- under the mutex, copy with `kfifo_to_user(&fifo, buf, n, &copied)`; return its error or `copied`;
- if another reader emptied the fifo first (`copied == 0`), wait again instead of returning 0.

NEED	API
IRQ	`request_irq`, `free_irq`, `irqreturn_t`, `IRQ_HANDLED` / `IRQ_NONE`
byte queue	`DEFINE_KFIFO`, `kfifo_put`, `kfifo_is_empty`, `kfifo_to_user` (`<linux/kfifo.h>`)
sleep / wake	`DECLARE_WAIT_QUEUE_HEAD`, `wait_event_interruptible`, `wake_up_interruptible`

B · Checkpoint

```
# Guest
insmod /modules/lab-5-work.ko
M=$(awk '$2=="serialaos" {print $1}' /proc/devices)
mknod -m 600 /dev/serialaos c $M 0
grep serialaos /proc/interrupts
mknod -m 666 /dev/null c 1 3
cat /dev/serialaos & CAT=$!
```

The guest shell has no controlling terminal: `Ctrl-C` does not reach `cat`, so run it in the background (its stdin becomes `/dev/null`, missing in the initramfs). Type `ciao from the host` in terminal 2, then:

```
# Guest
grep serialaos /proc/interrupts
kill $CAT
```

Paste a 100-character line in terminal 2 with nobody reading, then `head -c 64 /dev/serialaos | wc -c` and `rmmod module`.

Expected: the line appears in the guest; IRQ 3 counts 19; `64`; `dropped=37`.

B · Questions

1. Why can the handler and `read` share the kfifo without a lock between them?
2. What is `IRQ_NONE` for?
3. Why 19 interrupts for `ciao from the host`?
4. Why not check the fifo and then go to sleep?
5. What happens when `kill` hits a `cat` waiting in `read`?
6. Why a mutex, and not a spinlock, around `kfifo_to_user`?
7. Why 37 bytes lost out of 101?

Part C — Use the driver

C · Task: profile the driver, export the profile

Close terminal 2 (nobody connected: QEMU discards the bytes). With the module loaded and the node created:

```
# Guest
mknod /dev/zero c 1 5
mkdir -p /tmp; export PERF_PAGER=cat
perf record -e cpu-clock -F 999 -a -g -o /tmp/tx.data \
  -- dd if=/dev/zero of=/dev/serialaos bs=4k count=64
perf script -i /tmp/tx.data > /tmp/tx.txt; cksum /tmp/tx.txt
```

Capture on the host **without** `-t` (a terminal would turn `\n` into `\r\n`):

```
# Host
docker exec -i lkp-dev-amd64 /staging/initramfs/fs/bin/busybox \
  nc 127.0.0.1 6000 > tx.txt
```

Then `cat /tmp/tx.txt > /dev/serialaos` in the guest.

C · Checkpoint

When `cksum tx.txt` on the host matches the guest, stop the capture and draw (scripts from Brendan Gregg's Flame Graphs):

```
# Host
./scripts/sc.pl tx.txt > tx.folded
./scripts/fg.pl tx.folded > tx.svg
```

Open `tx.svg` in a browser, or load `tx.txt` into speedscope. In our run 191 of 218 samples were in your `write` function (ours is `uart_write`).

Questions.

1. Why use COM2 instead of printing the profile on the console?
2. `dd` reports about 1.3 MB/s. A real 115200-baud line carries about 11.5 KB/s. Why the difference?
3. Where does the time in your `write` function go?

Wrap-up

Done when

- `/dev/serialaos` sends text to terminal 2 and `cat` prints what you type there;
- IRQ 3 counts one interrupt per received byte; `dropped` is reported on removal;
- after `rmmod module`, `/proc/ioports` and `/proc/interrupts` no longer show `serialaos`;
- your flame graph shows your `write` function on top of `dd`;
- `dmesg | grep -E 'BUG:|WARNING:|Oops:'` prints nothing.

Linkography

- https://linux-kernel-labs.github.io/refs/heads/master/labs/interrupts.html
- https://www.kernel.org/doc/html/latest/driver-api/basics.html
- https://www.kernel.org/doc/html/latest/locking/locktypes.html
- https://alexrhodes.io/blog/post/22/ (good tutorial, but code has some bugs)
- https://en.wikipedia.org/wiki/16550_UART
- https://github.com/fedetft/miosix-kernel/blob/master/miosix/arch/common/drivers/serial_lpc2000.cpp